

Computer Architecture: The Instruction Set Architecture (ISA) *Part 1: Generics*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How do you observe the machine running?
- **How do we define the machine?**

Defining a **Processor**

We will only need a few simple components:

- **Memories** – to store our program (instructions) and data
- **A register file** – instructions will read their operands from the register file and also write their results to it.
- **Registers, an ALU and adders**
- **Decode and control logic**

A Simple (32-bit) Processor

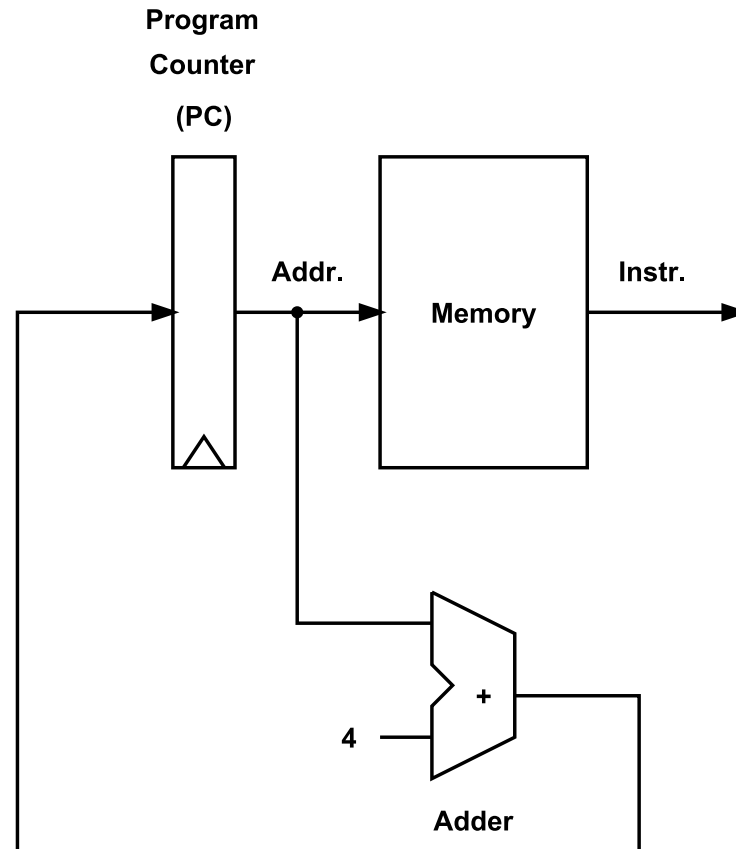
- Let's assume all our instructions are encoded in 32-bits/
- Our registers and datapath are also 32-bits wide.
- Memory is accessed with a 32-bit address and returns 32-bit data.
- Our processor has 32 registers, hence we must use 5-bits to identify a particular register (as $2^5 = 32$).

A Processor Datapath – PC and Instruction Memory

Our Program Counter (PC) stores the address of the instruction currently being fetched from memory.

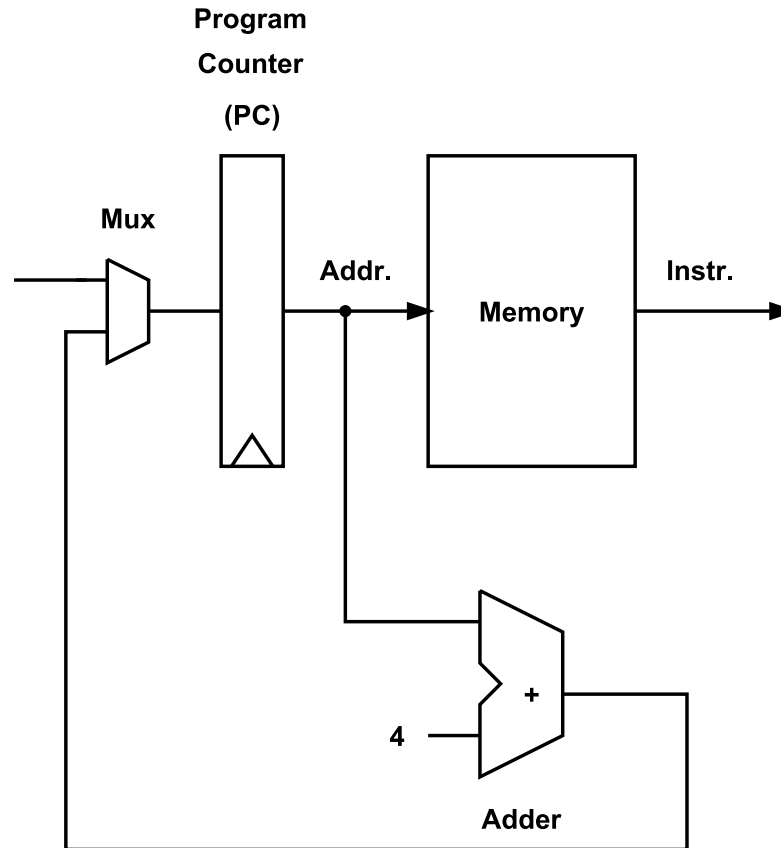
This simple datapath increases the PC by 4 (bytes) on each cycle and only allows us to read each 32-bit instruction in turn.

We need to add more logic to actually compute, handle branches, provide registers, access data memory, etc.



A Processor Datapath – Add mux to Allow Jump/Branch

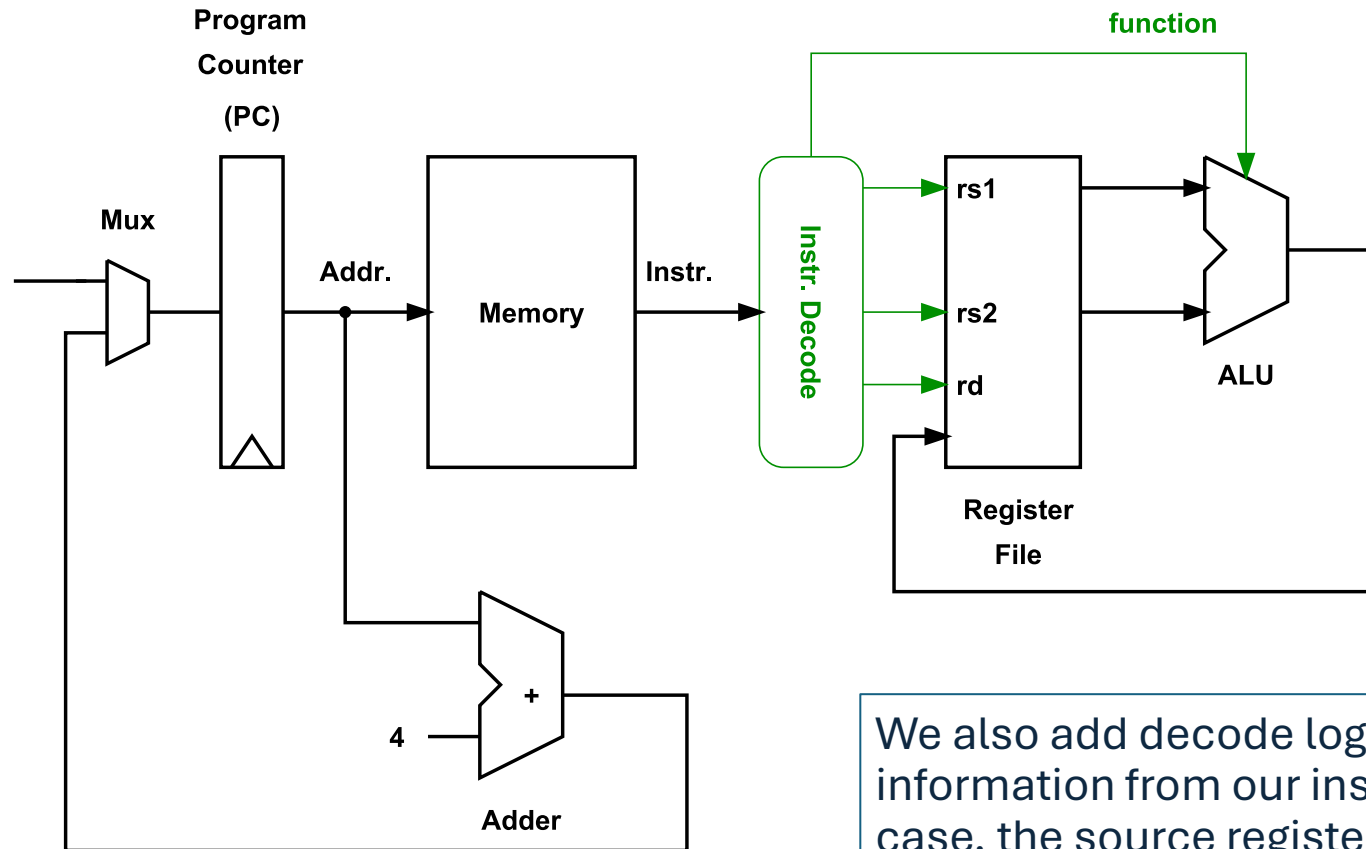
We add a multiplexor (mux) to allow us to provide a branch target address, i.e., the PC value following a taken branch.



A Processor Datapath – Add a Register File and ALU

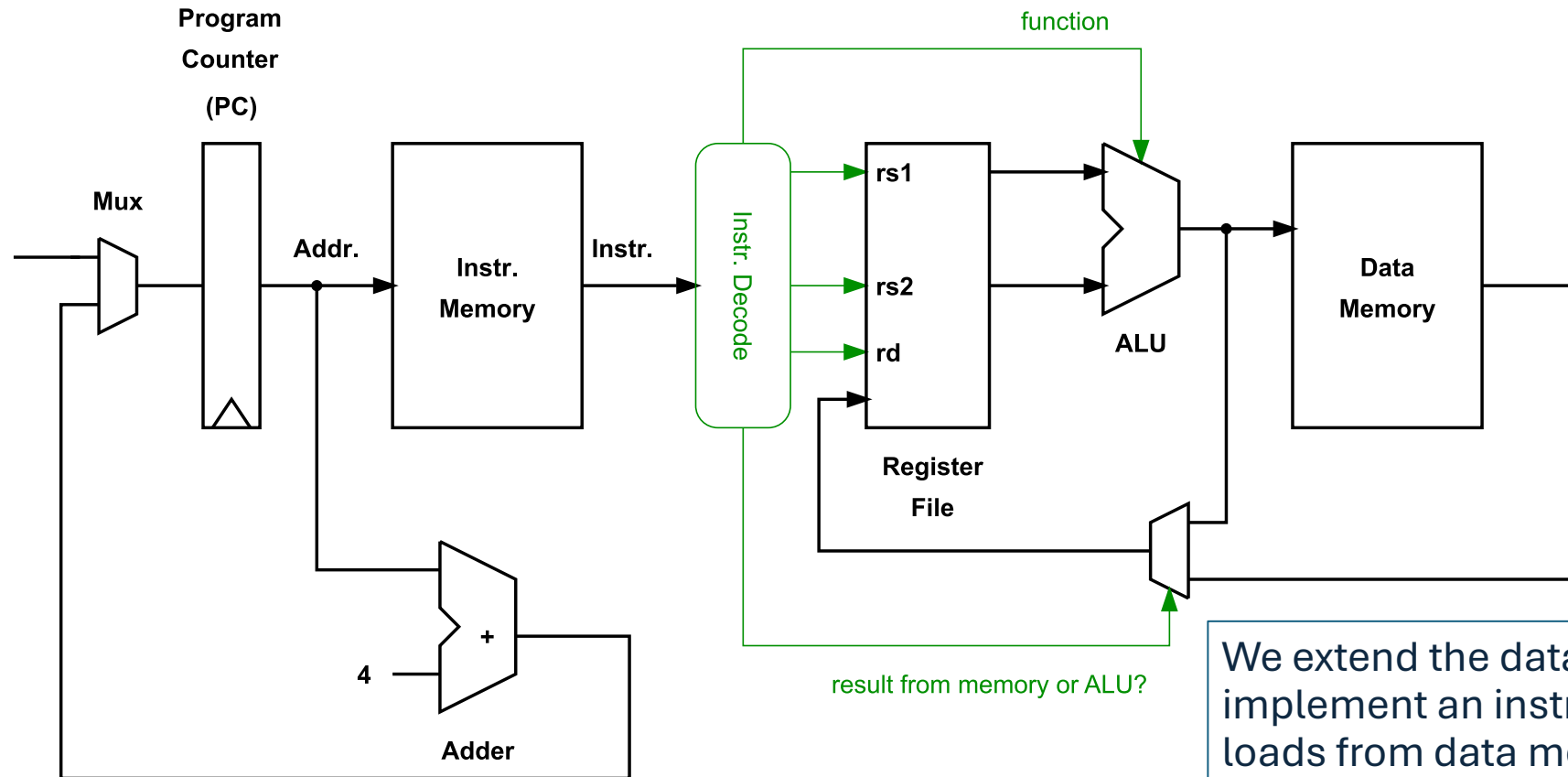
We could now execute a simple sequence of ALU operations.

Although, we do not yet have access to data memory and can't branch!



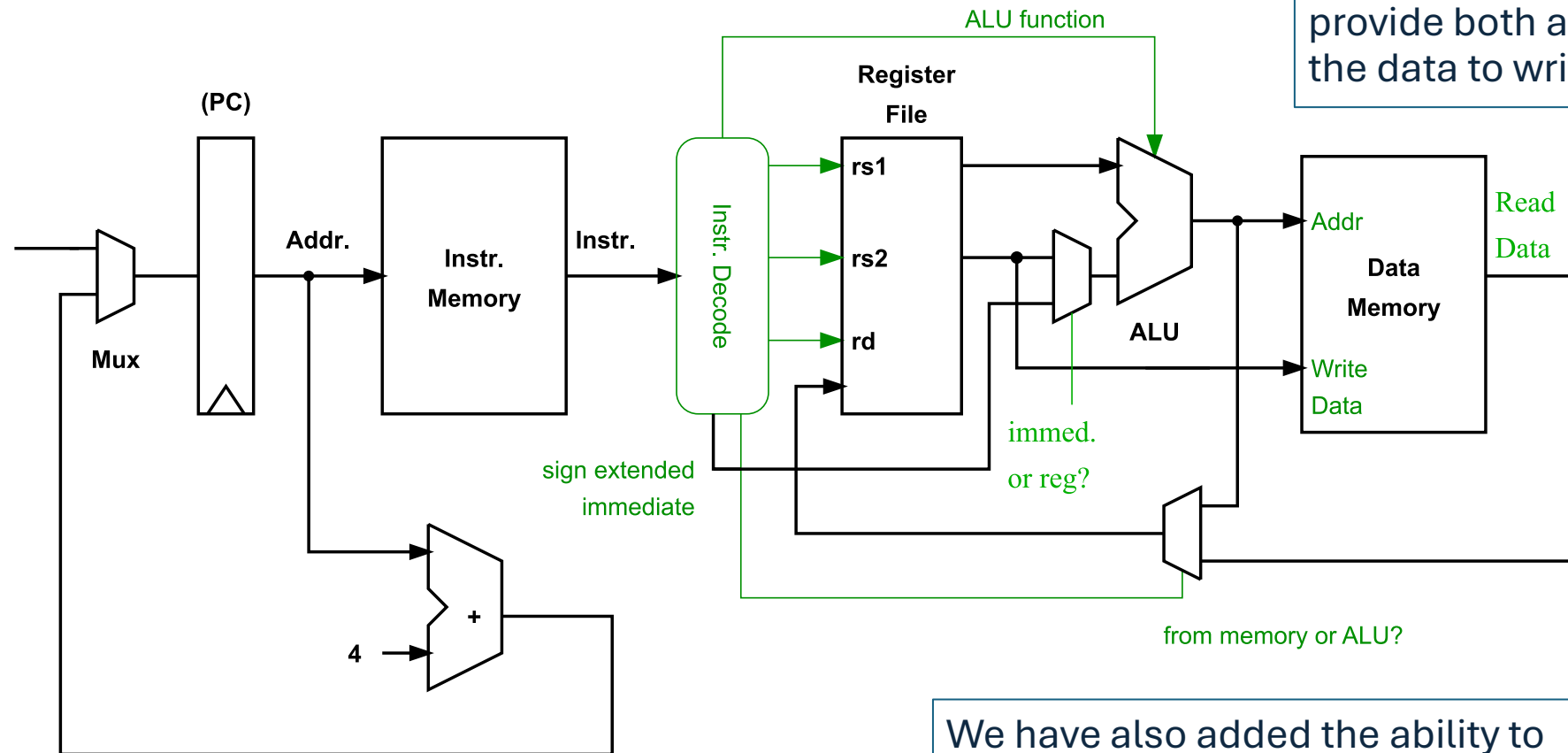
We also add decode logic that extracts information from our instruction; in this case, the source registers (rs1, rs2), destination register (rd), and the ALU function to be performed.

A Processor Datapath – Loading from Data Memory



We extend the datapath so we can implement an instruction that loads from data memory. The effective address could be the sum of the two source registers.

A Processor Datapath – Storing to Data Memory

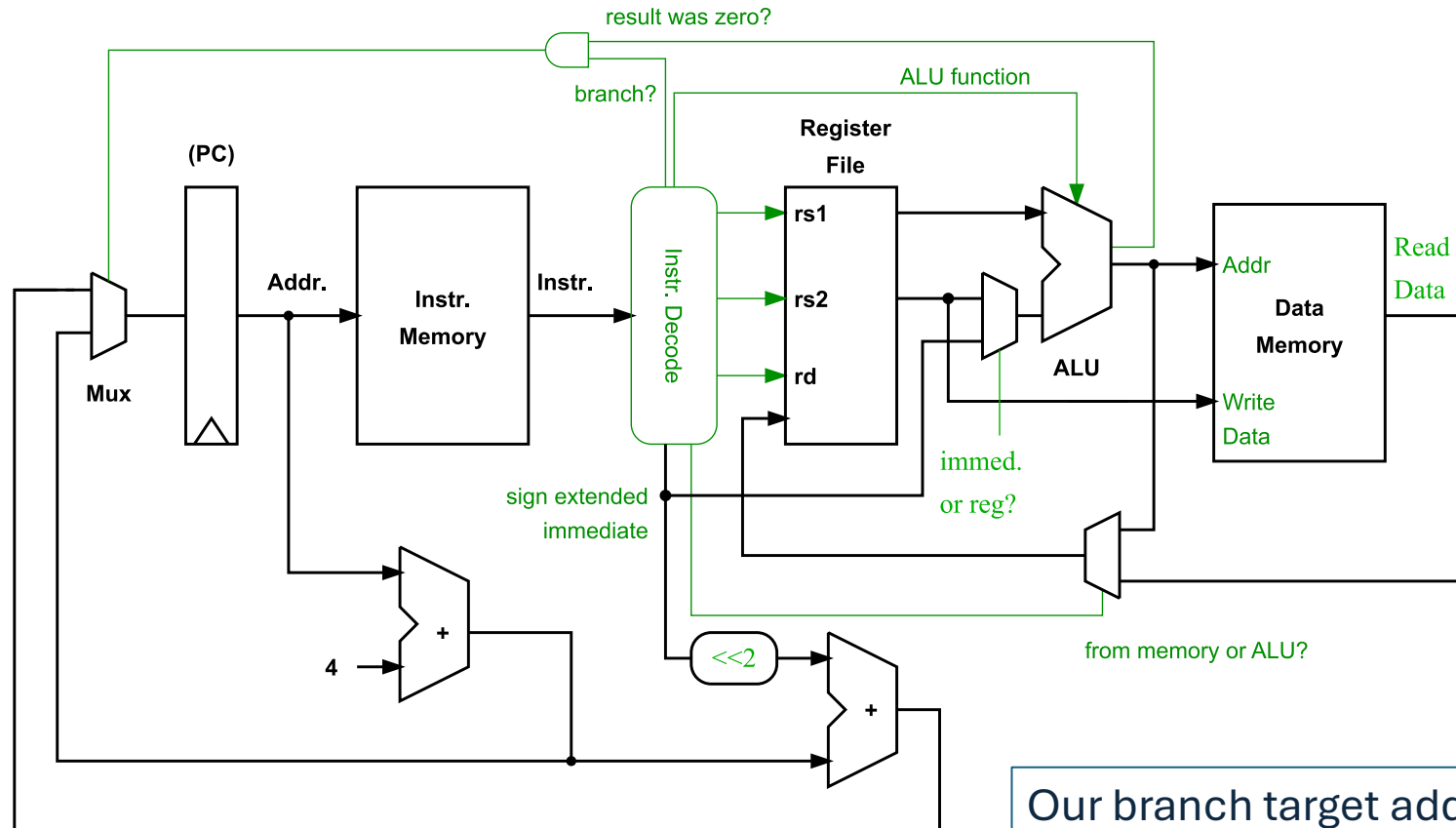


Storing to memory requires we provide both an address and the data to write.

We have also added the ability to extract immediate values from the instructions and use them in ALU operations or address calculations.

A Processor Datapath – Supporting Branch Instructions

Here, we assume a simple “branch if equal to zero” instruction.



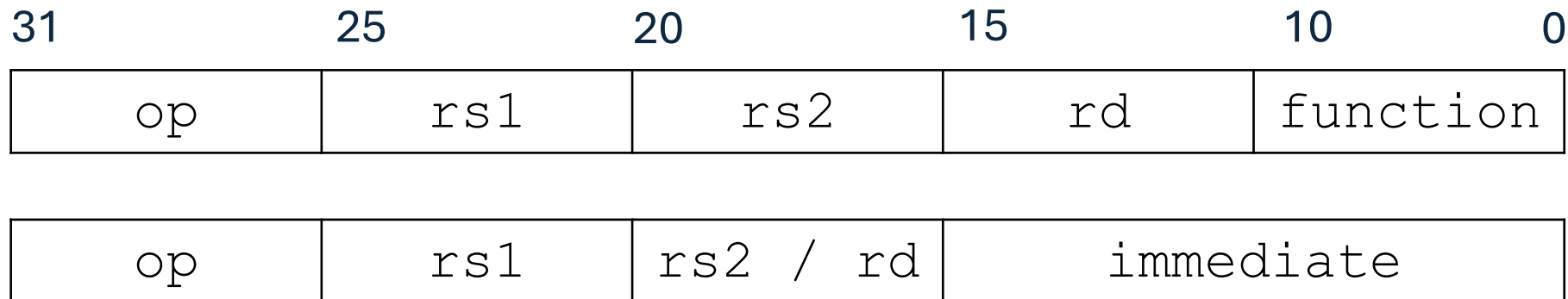
Our branch target address is computed relative to the PC. Our immediate (the offset) is shifted left by two (i.e., multiplied by 4) as all instructions are 32-bits.

Encoding our Instructions

- A simple data processing instruction may have the following format, where `Operand2` may be a register or immediate value.

`Instruction Rd, Rs, Operand2`

- Given 32-bits to encode our instructions, we may invent two simple instruction encoding formats for our processor, e.g.:



What the ISA?

- ISA Specifies:
 - Instruction encodings
 - Registers
 - Data types
 - Addressing modes
 - Memory models
 - Privilege levels
 - Exception behaviors

What the ISA?

- ISA **DOES NOT** Specify:
 - Clock speeds
 - Pipeline depth
 - Cache sizes
 - Power consumption
 - ... etc

ISA does not specify **HOW it works**

ISA Code Compatibility

- In general we look for code compatibility across the ISA.
- Code that works on one processor in the ISA should work on others.
- There may be certain features that can be enabled or disabled (will come back to this)

CISC vs. RISC

- **Complex Instruction Set Computer (CISC)**

- Do very complex tasks
- Most instructions (arithmetic) can access memory
- Variable-length instruction encoding (e.g., for x86 from 8 to 120 bits)
- x86 is a common example

- **Reduced Instruction Set Computer (RISC)**

- Do simple tasks – but do them quickly
- Only special load/store instructions can access memory
- Fixed-length instruction encoding (or a small number of choices, e.g., 32 or 16 bits)
- Arm is a common example

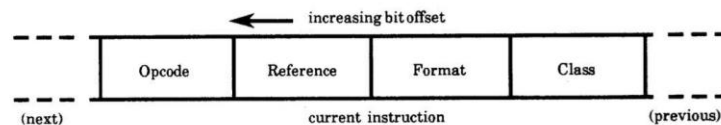
Sidenote: “Super-CISC” Architectures



iAPX432 Board breakdown CC BY SA 3.0

INSTRUCTION STREAM

Instructions are variable-length sections of a bit-addressed stream in an instruction object. The GDP interprets these instructions as being composed of fields of varying numbers of bits. The bit stream is scanned from lower-address bits to higher-address bits. Each field ends when the GDP recognizes a valid encoding; (one valid encoding is never the same as the beginning of another valid encoding;) the next bit in the stream begins the next field or next instruction. The fields are organized to present information to the processor in the sequence required for decoding. Every instruction contains an operator specification and possibly several references. The operator specifies to the processor what operation is to be performed, and the references specify the operands to be used or manipulated. The major fields of an instruction are ordered as follows:



F-0012

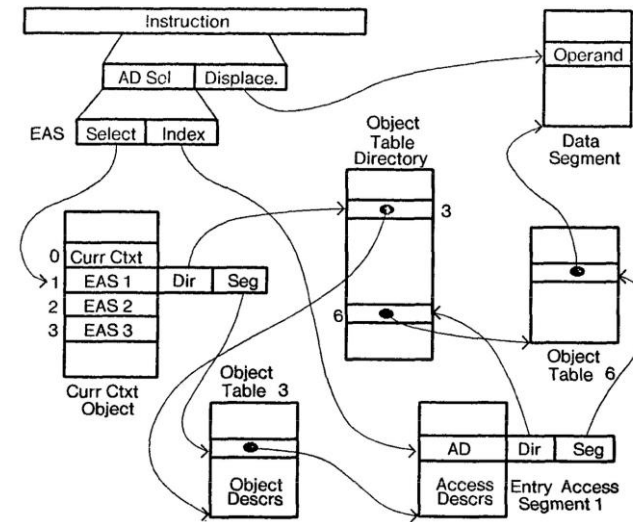
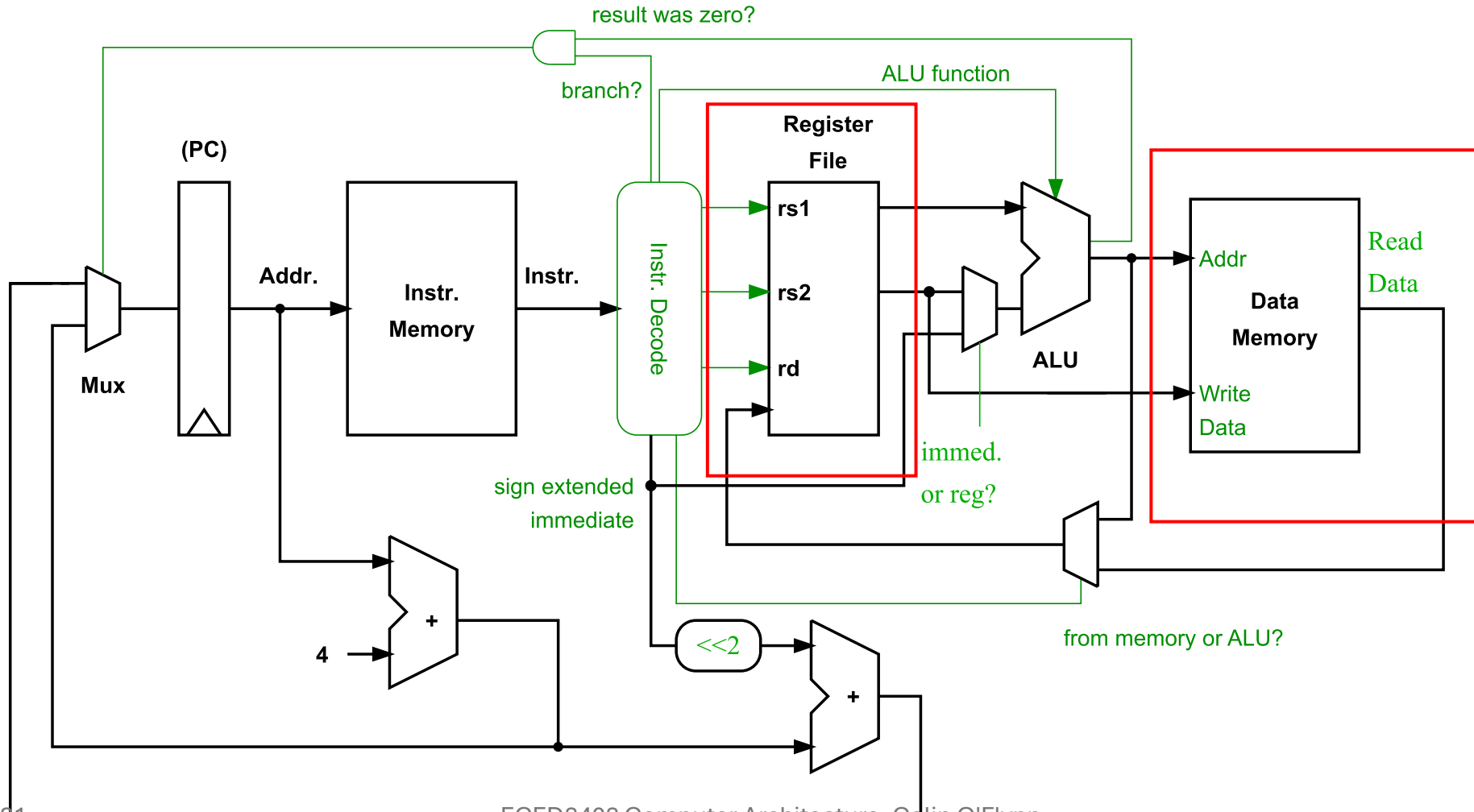


Fig. 2. The 432's full addressing path.

Registers & Memory



Register-Memory Architecture

- One of the operations of ALU instructions *may* be in memory
- Normal in CISC ISA, e.g., x86:

Instruction	Description	Pseudocode
<code>ADD eax, [memory_var]</code>	Adds the 32-bit value stored at the memory address <code>memory_var</code> to the EAX register. The result is stored in EAX.	$EAX = EAX + *(int*)memory_var$
<code>IMUL eax, [ebx + 4]</code>	Fetches a 32-bit value from memory at the address <code>ebx + 4</code> and multiplies it by the value in <code>eax</code> , storing the result back in <code>eax</code> .	$EAX = *(int*)(EBX+4) \times EAX$

Load-Store Architecture

- Also called register-register architecture
- Normal in RISC ISA
- Special load/store instructions are the only way to access memory
- E.g., Arm:

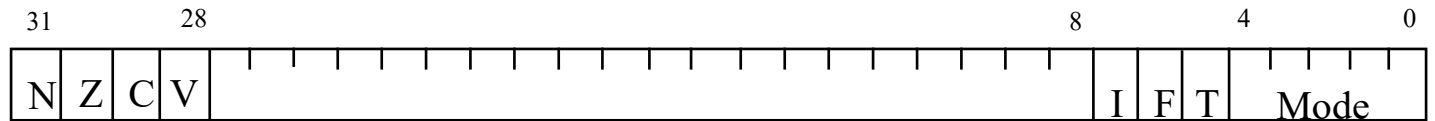
Instruction	Description	Pseudocode
<code>ldr r1, [r0]</code>	Load the value from the address in R0 into R1	$R1 = *((int*)R1)$

Registers and the ISA in ARM

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)

Program Status Registers

cpsr



Copies of the ALU status flags (latched if the instruction has the "S" bit set).

N = Negative result from ALU flag.
Z = Zero result from ALU flag.
C = ALU operation Carried out
V = ALU operation oVerflowed

Mode Bits

M[4:0] define the processor mode.

Interrupt Disable bits.

I = 1, disables the IRQ.
 F = 1, disables the FIQ.

T Bit (Architecture v4T only)

T = 0, Processor in ARM state
 T = 1, Processor in Thumb state

Privilege Levels & Operating Modes

- Most ISA have an idea of user execution and privileged execution
- Controls:
 - Access to registers
 - Access to memory
 - Possibly some instructions

Registers in Operating Modes

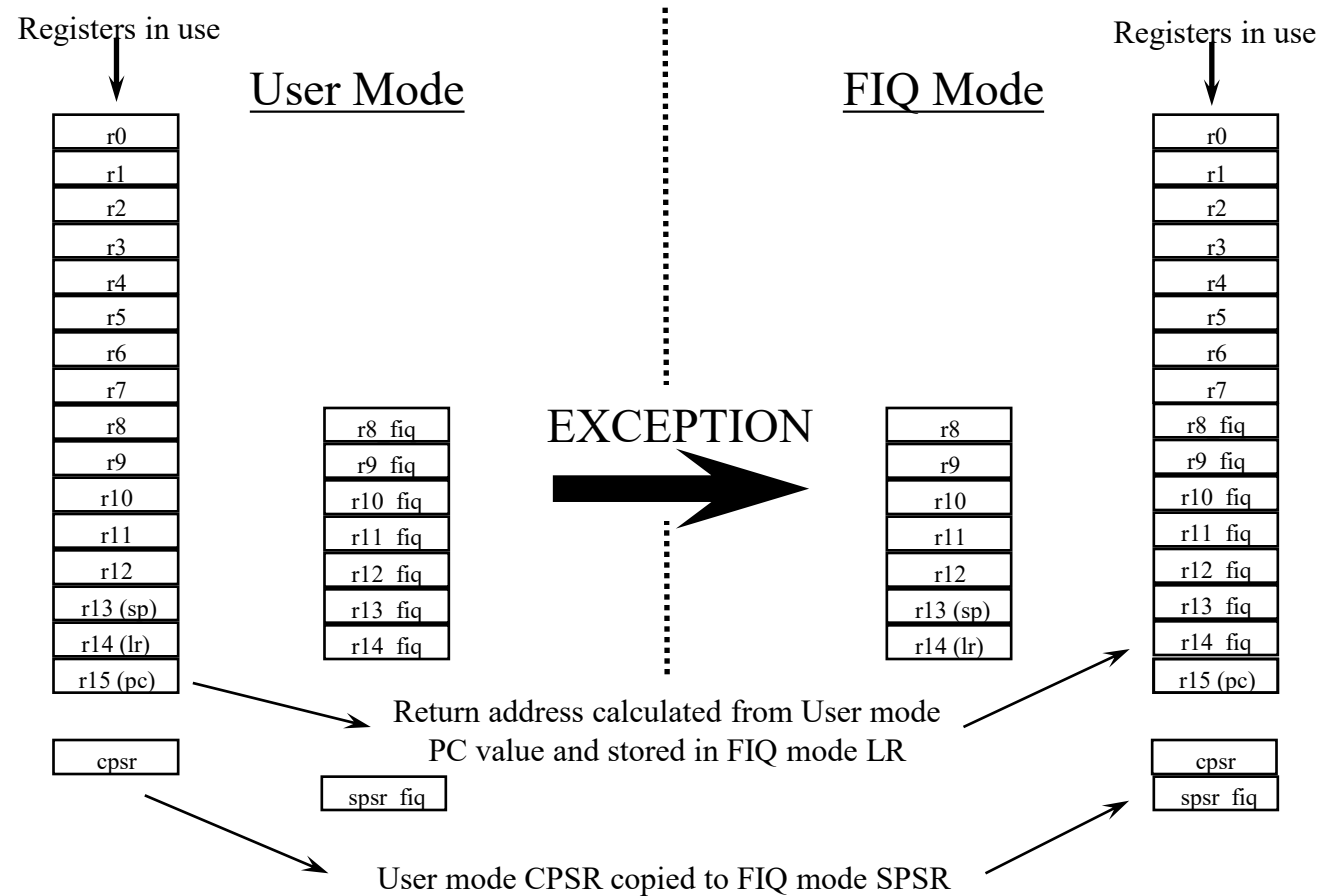
General registers and Program Counter

User32 / System	FIQ32	Supervisor32	Abort32	IRQ32	Undefined32
r0	r0	r0	r0	r0	r0
r1	r1	r1	r1	r1	r1
r2	r2	r2	r2	r2	r2
r3	r3	r3	r3	r3	r3
r4	r4	r4	r4	r4	r4
r5	r5	r5	r5	r5	r5
r6	r6	r6	r6	r6	r6
r7	r7	r7	r7	r7	r7
r8	r8 fiq	r8	r8	r8	r8
r9	r9 fiq	r9	r9	r9	r9
r10	r10 fiq	r10	r10	r10	r10
r11	r11 fiq	r11	r11	r11	r11
r12	r12 fiq	r12	r12	r12	r12
r13 (sp)	r13 fiq	r13 svc	r13 abt	r13 irq	r13 undef
r14 (lr)	r14 fiq	r14 svc	r14 abt	r14 irq	r14 undef
r15 (pc)	r15 (pc)	r15 (pc)	r15 (pc)	r15 (pc)	r15 (pc)

Program Status Registers

cpsr	cpsr	cpsr	cpsr	cpsr	cpsr
	spsr fiq	spsr svc	spsr abt	spsr irq	spsr undef

e.g., User to Fast Interrupt (FIQ)



Addressing Modes

- Register
- Register + offset
- Immediate
- PC-relative

Addressing: Register

- Operands in register, e.g.:

ADD r0, r1, r2 $\rightarrow$ $r0 = r1 + r2$

Addressing: Register + Offset (Base + Offset)

- Base register + immediate offset

`LDR r0, [r1, #8] → r0 = *(uint32_t *) (r1 + 8)`

- Consider:
 - C struct fields
 - Stack pointer or frame

Addressing: Stack-Based Access

- Base register + offset:

`LDR r0, [sp, #12] → r0 = *(uint32_t *) (sp + 12)`

- Remember that “sp” is another register – this makes our usage of the stack very easy (or the compiler’s usage)

Addressing: Register Offset Addressing

- Base register + offset stored in another register

`LDR r0, [r1, r2] → r0 = *(uint32_t *) (r1 + r2)`

- Array indexing & pointer arithmetic:

`arr[i]`

Addressing: Scaled Register Offset

- Offset scaled by element size

`LDR r0, [r1, r2, LSL #2] →  $r0 = *(uint32\_t *) (r1 + r2 * 4)$`

LSL = Logical Shift Left:

`LSL #1 =  $a \ll 1 = a \times 2$`

`LSL #2 =  $a \ll 2 = a \times 4$`

`LSL #3 =  $a \ll 3 = a \times 8$`

Makes for efficient table & array indexing!

Addressing: Immediate

- Operand is encoded directly in the instruction, e.g.:

`ADD r0, r1, #5` $\rightarrow$ `r0 = r1 + 5`

Constraint: **Operand size is limited**, or you have weird encodings
(*we will see this later when we go through Arm instruction set*)

Why? Remember we want **fixed instruction lengths**.

Addressing: Immediate (Cont'd)

e.g., using Immediate Addressing to load a larger constant:

```
MOVW  r0, #0x1234    ; Move Wide (lower 16 bits)
MOVT  r0, #0x5678    ; Move Top   (upper 16 bits)
```

...ARM uses a more complex encoding even that means these won't work as-is necessarily, more on that later...

Addressing: PC-Relative

- Address = PC + Offset:

```
LDR r0, [pc, #24] → r0 = *(uint32_t *) (pc + 24)
```

Load *relative to the current code location*. This is a powerful tool as it means *if you move the code & constants together, the addresses are all still accurate*.

This is used for loading constants in flash normally, which get placed near the code they are used in.

Addressing: PC-Relative Branches

- Not data, but we have the same thing for the “branch” (control flow) instruction:

B #64 $\rightarrow$ $pc = pc + 64$

Relative means code can be relocated without recalculating branches.

Addressing: Pre-Indexed Addressing

Address is updated before access. E.g.:

```
LDR r0, [r1, #4] !
```

Is decoded to:

$$r1 = r1 + 4$$
$$r0 = * (r1)$$

Very handy for pointer incrementing!

Addressing: Post-Indexed Addressing

Address is updated after access. E.g.:

```
LDR r0, [r1], #4
```

Is decoded to:

$$r0 = * (r1)$$
$$r1 = r1 + 4$$

Very handy for copy loops!

Memory Model

- *Addressing mode: how an address is computed*
- *Memory model: how memory operations are ordered in execution*
- **Sequentially Consistent**
 - Everything happens in program order
 - Slower (can get caught on one slow execution)
- **Relaxed Model**
 - Reordering allowed
 - Higher performance

Arm Memory Model

- ARM is *explicitly* relaxed (most modern ISAs are relaxed)
- Special care taken when order matters:
 - “Barriers” are an instruction that tells arm to ensure memory operations complete before continuing execution
 - Atomic operations guarantee a complete write or read happens

Odd ISA Instructions?

ISAs try to encode common patterns, not just arbitrary math.

- You may run across “weird” instructions, like:
 - Bit-field operations (set & clear bits)
 - Multiply-accumulate
 - Special barrier instructions
- These are normally designed to optimize common embedded use-cases, like:
 - Setting/clearing bits in registers
 - Enabling efficient compiler patterns
 - For specific power or timing reasons

The ARM ISA Family

- **(A)**pplication
 - **(R)**eal-Time
 - **(M)**icrocontroller
-
- Arm Cortex-A = Application Level (Cell Phone)
 - Arm Cortex-M = Microcontroller (what we'll look at)
 - Arm Cortex-R = Real-time specific (similar to microcontroller)

ISA Code Compatibility – e.g., Arm ISA

Feature	M0 / M0+	M3	M4	M7	M23	M33
Thumb (16-bit)	Yes	Yes	Yes	Yes	Yes	Yes
Thumb-2 (32-bit)	No	Yes	Yes	Yes	No	Yes
MOVW / MOVT	No	Yes	Yes	Yes	No	Yes
ISA version	ARMv6-M	ARMv7-M	ARMv7E-M	ARMv7E-M	ARMv8-M	ARMv8-M

ISA Code Compatibility – e.g., Arm Optional

Feature	M0 / M0+	M3	M4	M7	M23	M33
DSP instructions	No	No	Yes	Yes	No	Optional
Hardware divide	No	Optional	Optional	Optional	No	Optional
Single-precision FPU	No	No	Optional	Optional	No	Optional
Double-precision FPU	No	No	No	Optional	No	No
TrustZone-M	No	No	No	No	Yes	Yes

Summary

- Instruction Set Architecture defines the machine:
 - Memory model
 - Addressing
 - Registers
 - Instructions
- There are different design choices, most break down to:
 - Register-Memory (CISC)
 - Register-Register / Load-Store (RISC)
- Reviewed common addressing modes
- Reviewed examples of code compatibility on Cortex-M specifically